UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

CORSO DI LAUREA IN INGEGNERIA INFORMATICA
CORSO DI INGEGNERIA DEL SOFTWARE
PROF. S. RUSSO - A.A. 2022 - 23

Progetto

Biglietteria compagnia di navigazione

Gruppo: Tre uomini e un progetto

Studenti: Luciano Vitale N46004566 luciano.vitale@studenti.unina.it

Indice

1. Specifiche Informali 4

2. Analisi e specifica dei requisiti 5

2.1 Analisi nomi – verbi 5

2.2 Revisione dei requisiti 6

2.3 Glossario dei termini 6

2.4 Classificazione dei requisiti 7

2.4.1 Requisiti funzionali 7

2.4.2 Requisiti sui dati 8

2.4.3 Vincoli / Altri requisiti 8

2.5 Modellazione dei casi d'uso 9

2.5.1 Attori e casi d'uso 9

2.5.2 Diagramma dei casi d'uso 10

2.5.3 Scenari 11

2.6 Modellazione dei dati 15

2.6.1 Progettazione concettuale 15

2.7 Diagramma delle classi 16

2.8 Diagrammi di sequenza 18

2.9 Verifica della completezza dei requisiti 19

3. Stima dei costi 20

4. Piano di test funzionale 22

5. Progettazione 26

5.1 Progettazione della base di dati 26

5.1.1 Progettazione logica 26

5.2 Diagramma delle classi 28
5.3 Diagramma di sequenza 29

6. Implementazione 30

7. Testing 31
7.1 Test Strutturale 31
7.1.1 Complessità ciclomatica 31
7.1.2 Test di unità 32
7.2 Test funzionale 32

1. Specifiche Informali

Si vuole realizzare un'applicazione per compagnie di navigazione per l'emissione dei biglietti per viaggi marittimi.
Una compagnia di navigazione (caratterizzata dal proprio nome) fornisce servizi di trasporto passeggeri (operati con aliscafi e traghetti) e di trasporto passeggeri e autoveicoli (operati con traghetti). Ogni nave ha un nome e un numero di posti disponibili per i passeggeri e, per i traghetti, anche una capienza per gli autoveicoli.

Una compagnia di navigazione opera con le proprie navi un insieme di corse giornaliere tra N porti. Esistono due tipi di biglietti, per passeggero e per autoveicolo. Ogni corsa è identificata da un codice e ha orario e porto di partenza e di arrivo, e un prezzo dei biglietti prefissato. Una corsa è effettuata sempre dalla stessa nave.

La biglietteria di una compagnia può emettere biglietti viaggio, validi per una specifica corsa in un determinato giorno. Prima di emettere un biglietto, il sistema deve valutare l'effettiva disponibilità del posto. All'atto dell'emissione dovrà essere aggiornata la disponibilità residua.
L'applicazione supporta l'emissione di biglietti da parte degli impiegati di ciascuna compagnia. Ogni impiegato può accedere al sistema solo previa autenticazione. Per ogni biglietto il sistema deve stampare la data e l'ora di emissione. Per ciascun biglietto il sistema deve registrare internamente, al momento dell'emissione, data e ora di emissione e codice dell'impiegato che lo ha emesso. Per i biglietti per i veicoli occorre registrare e stampare la targa del veicolo.

È altresì previsto l'uso dell'applicazione da parte di utenti clienti di una compagnia, che possono prenotare via web un biglietto passeggero e/o per autoveicolo. L'emissione del biglietto prenotato avverrà tuttavia a cura di un impiegato della compagnia, cui il cliente dovrà esibire una stampa della prenotazione che riporti gli estremi della corsa e, eventualmente, la targa del veicolo.

2. Analisi e specifica dei requisiti

2.1 Analisi nomi – verbi

Si vuole realizzare un'applicazione per compagnie di navigazione per l'emissione dei biglietti per viaggi marittimi.
Una compagnia di navigazione (caratterizzata dal proprio nome) fornisce servizi di trasporto passeggeri (operati con aliscafi e traghetti) e di trasporto passeggeri e autoveicoli (operati con traghetti). Ogni nave ha un nome e un numero di posti disponibili per i passeggeri e, per i traghetti, anche una capienza per gli autoveicoli.

Una compagnia di navigazione opera con le proprie navi un insieme di corse giornaliere tra N porti. Esistono due tipi di biglietti, per passeggero e per autoveicolo. Ogni corsa è identificata da un codice e ha orario e porto di partenza e di arrivo, e un prezzo dei biglietti prefissato. Una corsa è effettuata sempre dalla stessa nave.

La biglietteria di una compagnia può emettere biglietti viaggio, validi per una specifica corsa in un determinato giorno. Prima di emettere un biglietto, il sistema deve valutare l'effettiva disponibilità del posto. All'atto dell'emissione dovrà essere aggiornata la disponibilità residua.
L'applicazione supporta l'emissione di biglietti da parte degli impiegati di ciascuna compagnia. Ogni impiegato può accedere al sistema solo previa autenticazione. Per ogni biglietto il sistema deve stampare la data e l'ora di emissione. Per ciascun biglietto il sistema deve registrare internamente, al momento dell'emissione, data e ora di emissione e codice dell'impiegato che lo ha emesso. Per i biglietti per i veicoli occorre registrare e stampare la targa del veicolo.

È altresì previsto l'uso dell'applicazione da parte di utenti clienti di una compagnia, che possono prenotare via web un biglietto passeggero e/o per autoveicolo. L'emissione del biglietto prenotato avverrà tuttavia a cura di un impiegato della compagnia, cui il cliente dovrà esibire una stampa della prenotazione che riporti gli estremi della corsa e, eventualmente, la targa del veicolo.

LEGENDA:

Classe
Attributo
Funzionalità
Attore
Classe-Attore

2.2 Revisione dei requisiti

1. *Il servizio di trasporto passeggeri è operato con aliscafi e traghetti.*
2. *Il servizio di trasporto passeggeri e autoveicoli è operato con traghetti.*
3. *Ogni nave ha un nome e un numero di posti disponibili per i passeggeri.*
4. *Ogni traghetto ha anche una capienza per gli autoveicoli.*
5. *Ogni nave effettua varie corse giornaliere.*
6. *Una corsa opera tra due porti.*
7. *Una corsa è effettuata sempre dalla stessa nave*
8. *Ogni corsa è identificata da un codice.*
9. *Ogni corsa ha un orario e porto di partenza e di arrivo, e un prezzo dei biglietti.*
10. *Il sistema deve offrire ai clienti una funzionalità per prenotare via web un biglietto.*
11. *Esistono due tipi di biglietto, per passeggero e per autoveicolo.*
12. *I biglietti sono validi per una specifica corsa in un determinato giorno.*
13. *Il sistema deve offrire ai clienti una funzionalità per ricevere una stampa della prenotazione che riporti gli estremi della corsa e, eventualmente, la targa dell'autoveicolo.*
14. *L'emissione dei biglietti è effettuata da parte degli impiegati di ciascuna compagnia.*
15. *Il sistema deve offrire agli impiegati della biglietteria una funzionalità per emettere biglietti viaggio, sia prenotati via web che non.*
16. *Il sistema deve offrire all'impiegato una funzionalità per autenticarsi.*
17. *Il sistema deve offrire una funzionalità per valutare l'effettiva disponibilità del posto.*
18. *Il sistema deve aggiornare la disponibilità dei posti residua ad ogni emissione di un biglietto.*
19. *Il sistema deve registrare internamente, al momento dell'emissione di ogni biglietto, data e ora di emissione e codice dell'impiegato che lo ha emesso e, eventualmente, la targa dell'autoveicolo.*
20. *Il sistema deve stampare per ogni biglietto la data e l'ora di emissione e, eventualmente, la targa dell'autoveicolo.*

2.3 Glossario dei termini

Termine	Descrizione	Sinonimi
Traghetto	Nave adibita al trasporto di passeggeri e/o autoveicoli	Nave
Aliscafo	Nave adibita al trasporto di passeggeri	Nave
Capienza	Numero di posti disponibili per passeggeri o per autoveicoli	
Corsa	Tratta effettuata da una nave tra due porti	

Prenotazione	Ricevuta, ottenuta via web, che consente al cliente di prenotare un biglietto per passeggero o per autoveicolo su una nave.	
Impiegato	Addetto della compagnia che si occupa dell'emissione di biglietti nella biglietteria.	
Cliente	Persona che effettua una prenotazione via web	Utente

2.4 Classificazione dei requisiti

2.4.1 Requisiti funzionali

ID	Requisito	Origine (n. frase dei requisiti revisionati)
RF01	Il sistema deve offrire una funzionalità per valutare l'effettiva disponibilità del posto	17
RF02	Il sistema deve offrire ai clienti una funzionalità per prenotare via web un biglietto	10
RF03	Il sistema deve offrire ai clienti una funzionalità per ricevere una stampa della prenotazione che riporti gli estremi della corsa e, eventualmente, la targa dell'autoveicolo.	13
RF04	Il sistema deve offrire agli impiegati della biglietteria una funzionalità per emettere biglietti viaggio prenotati via web.	15
RF05	Il sistema deve offrire all'impiegato una funzionalità per autenticarsi.	16
RF06	Il sistema deve offrire agli impiegati della biglietteria una funzionalità per emettere biglietti viaggio.	15
RF07	Il sistema deve aggiornare la disponibilità dei posti residua ad ogni emissione di un biglietto.	18
RF08	Il sistema deve registrare internamente, al momento dell'emissione di ogni biglietto, data e ora di emissione e codice dell'impiegato che lo ha emesso e, eventualmente, la targa dell'autoveicolo.	19
RF09	Il sistema deve stampare per ogni biglietto la data e l'ora di emissione e, eventualmente, la targa dell'autoveicolo.	20

2.4.2 Requisiti sui dati

ID	Requisito	Origine (n. frase dei requisiti revisionati)
RD01	Il servizio di trasporto passeggeri è operato con aliscafi e traghetti.	1
RD02	Il servizio di trasporto passeggeri e autoveicoli è operato con traghetti.	2
RD03	Ogni nave ha un nome e un numero di posti disponibili per i passeggeri.	3
RD04	Ogni traghetto ha anche una capienza per gli autoveicoli.	4
RD05	Ogni nave effettua varie corse giornaliere.	5
RD06	Una corsa opera tra due porti.	6
RD07	Ogni corsa è identificata da un codice	8
RD08	Ogni corsa ha un orario e porto di partenza e di arrivo, e un prezzo dei biglietti.	9
RD09	Esistono due tipi di biglietto, per passeggero e per autoveicolo.	11
RD10	I biglietti sono validi per una specifica corsa in un determinato giorno	12

2.4.3 Vincoli / Altri requisiti

ID	Requisito	Origine (n. frase dei requisiti revisionati)
V01	Una corsa è effettuata sempre dalla stessa nave	7
V02	Le corse di una compagnia sono giornaliere	5
RNF01	Per registrare i dati delle emissioni deve essere disponibile un server	19

2.5 Modellazione dei casi d'uso

2.5.1 Attori e casi d'uso

Attori primari:

- Impiegato
- Cliente

Casi d'uso:

- UC1: EmettiBiglietto
- UC2: EmettiBigliettoBiglietteria
- UC3: EmettiBigliettoPrenotato
- UC4: PrenotaBiglietto
- UC5: VisualizzaCorse

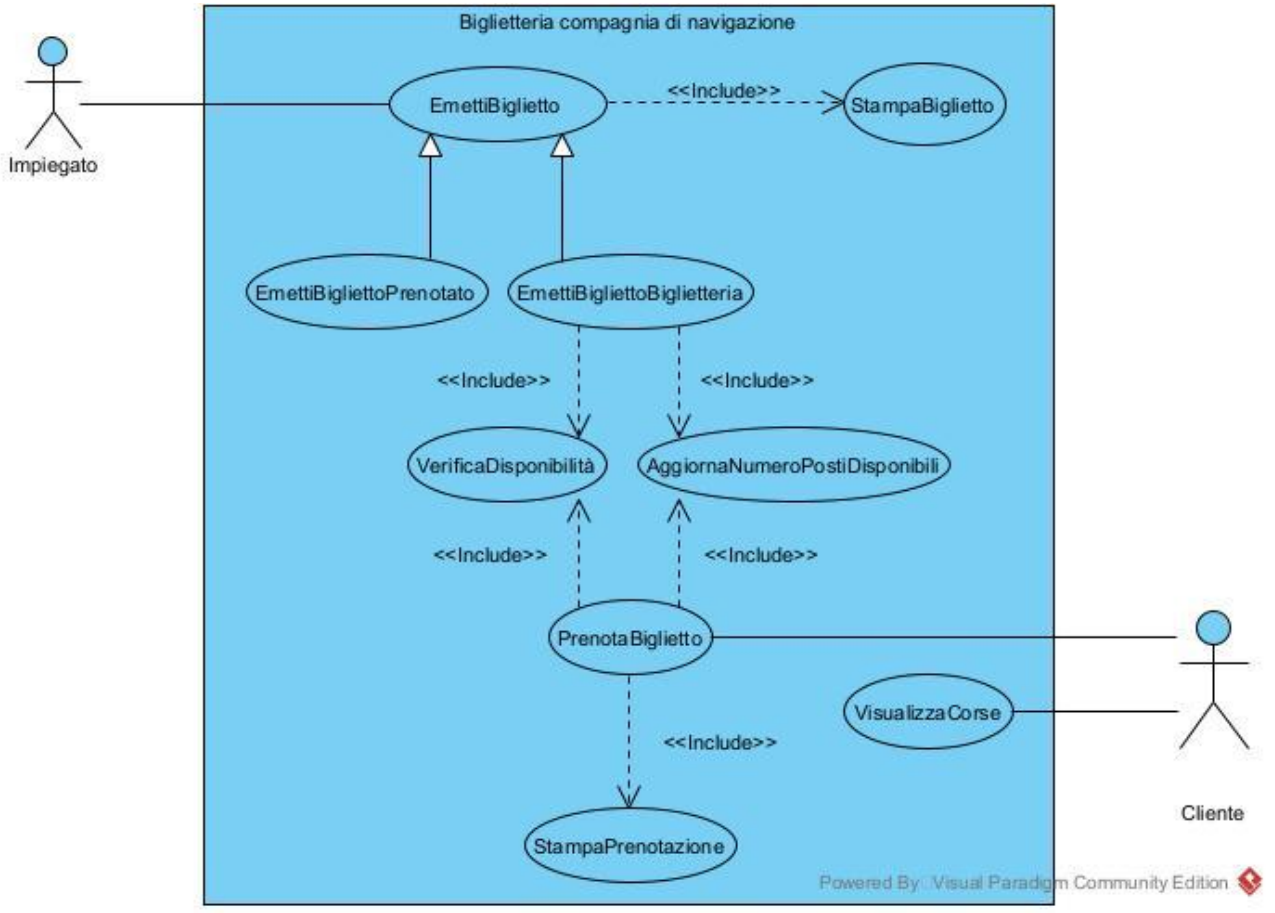
Casi d'uso d'inclusione:

- UC6: StampaBiglietto
- UC7: VerificaDisponibilità
- UC8: AggiornaNumeroPostiDisponibili
- UC9: StampaPrenotazione

Caso d'uso	Attori Primari	Attori Secondari	Incl. / Ext.	Requisiti corrispondenti
UC1: EmettiBiglietto	Impiegato	-	Generalizzazione di EmettiBigliettoBiglietteria e EmettiBigliettoPrenotato Include Stampa biglietto	RF04, RF05, RF06, RF08
UC2: EmettiBigliettoBiglietteria	Impiegato	-	Specializzazione di EmettiBiglietto Include VerificaDisponibilità AggiornaNumeroPostiDisponibili	RF05, RF06, RF08
UC3: EmettiBigliettoPrenotato	Impiegato	-	Specializzazione di EmettiBiglietto	RF04, RF05, RF08
UC4: PrenotaBiglietto	Cliente	-	Include StampaPrenotazione, VerificaDisponibilità e AggiornaNumeroPostiDisponibili	RF02
UC5: VisualizzaCorse	Cliente	-	-	RF02
UC6: StampaBiglietto	-	-	Incluso in EmettiBiglietto	RF09

UC7: VerificaDisponibilità	-	-	Incluso in EmettiBigliettoBiglietteria e PrenotaBiglietto	RF01
UC8: AggiornaNumeroPostiDisponibili	-	-	Incluso in EmettiBigliettoBiglietteria e PrenotaBiglietto	RF07
UC9: StampaPrenotazione	-	-	Incluso in PrenotaBiglietto	RF03

2.5.2 Diagramma dei casi d'uso



2.5.3 Scenari

Caso d'uso:	EmettiBigliettoPrenotato
Attore primario	Impiegato
Attore secondario	-
Descrizione	L'impiegato emette un biglietto prenotato via web valido per una corsa.
Pre-Condizioni	L'impiegato deve essersi autenticato nel sistema. Il cliente deve avere la stampa della prenotazione.
Sequenza di eventi principale	1. Il caso d'uso inizia quando l'impiegato inserisce nel sistema il codice della prenotazione. 2. Se il codice non è valido 2.1. Il sistema restituisce un messaggio di ERRORE. 3. Il sistema genera i biglietti validi per la corsa specificata nella prenotazione. 4. Il sistema registra internamente data e ora di emissione e codice dell'impiegato che lo ha emesso. 5. Se il biglietto è per autoveicolo 5.1. Il sistema registra internamente anche la targa dell'autoveicolo 6. Include (StampaBiglietto).
Post-Condizioni	Il sistema ha emesso un biglietto valido per una corsa. I dati dell'emissione vengono salvati nel sistema.
Casi d'uso correlati	StampaBiglietto
Sequenza di eventi alternativi	-

Caso d'uso:	EmettiBigliettoBiglietteria
Attore primario	Impiegato
Attore secondario	-
Descrizione	L'impiegato emette un biglietto valido per una corsa.
Pre-Condizioni	L'impiegato deve essersi autenticato nel sistema.
Sequenza di eventi principale	1. Il caso d'uso inizia quando l'impiegato richiede l'emissione di un biglietto. 2. L'impiegato specifica la tipologia del biglietto. 3. L'impiegato immette il giorno, l'orario, porto di partenza e di arrivo della corsa. 4. Il sistema controlla che la corsa esista per i dati specificati. 4.1. Se la corsa non esiste il sistema restituisce un messaggio di ERRORE. 5. Include (VerificaDisponibilità). 6. Se il biglietto è per autoveicolo 6.1. L'impiegato inserisce la targa dell'autoveicolo. 7. Il sistema mostra all'impiegato il costo del biglietto della corsa. 8. L'impiegato conferma l'emissione ed effettua il pagamento. 9. Il sistema genera un biglietto valido per la corsa specificata. 10. Include (AggiornaNumeroPostiDisponibili). 11. Il sistema registra internamente data e ora di emissione e codice dell'impiegato che lo ha emesso. 12. Se il biglietto è per autoveicolo 12.1. Il sistema registra internamente anche la targa dell'autoveicolo 13. Include (StampaBiglietto).
Post-Condizioni	Il sistema ha emesso un biglietto valido per una corsa. I dati dell'emissione vengono salvati nel sistema.
Casi d'uso correlati	VerificaDisponibilità AggiornaNumeroPostiDisponibili StampaBiglietto

Sequenza di eventi alternativi	-

Caso d'uso:	StampaBiglietto
Attore primario	-
Attore secondario	-
Descrizione	Il sistema stampa un biglietto valido per una corsa
Pre-Condizioni	Deve essere stato emesso un biglietto
Sequenza di eventi principale	1. Il caso d'uso inizia quando il sistema registra internamente i dati relativi al biglietto. 2. Il sistema effettua la stampa del biglietto registrato specificando data e ora di emissione. 3. Se il biglietto è per autoveicolo 3.1. Il sistema stampa anche la targa dell'autoveicolo
Post-Condizioni	Il sistema ha stampato un biglietto valido per una corsa
Casi d'uso correlati	EmettiBiglietto
Sequenza di eventi alternativi	-

Caso d'uso:	VerificaDisponibilità
Attore primario	-
Attore secondario	-
Descrizione	Il sistema effettua un controllo dei posti disponibili per la corsa specificata
Pre-Condizioni	L'impiegato o il cliente deve aver immesso nel sistema i dati per la corsa.
Sequenza di eventi principale	1. Il caso d'uso inizia quando il sistema ha controllato che la corsa esista per i dati specificati. 2. Il sistema verifica che ci siano ancora posti disponibili per la corsa specificata. 3. Se non ci sono posti disponibili 3.1. Il sistema restituisce un messaggio di ERRORE
Post-Condizioni	-
Casi d'uso correlati	EmettiBigliettoBiglietteria PrenotaBiglietto
Sequenza di eventi alternativi	-

Caso d'uso:	AggiornaNumeroPostiDisponibili
Attore primario	-
Attore secondario	-
Descrizione	Il sistema aggiorna il numero dei posti disponibili per la corsa specificata
Pre-Condizioni	Il sistema deve aver generato un biglietto non prenotato o una prenotazione valida per una corsa
Sequenza di eventi principale	1. Il caso d'uso inizia quando il sistema genera un biglietto non prenotato o una prenotazione valida per la corsa specificata. 2. Il sistema aggiorna il numero di posti disponibili per passeggeri.
Post-Condizioni	Il numero dei posti disponibili per la corsa è stato aggiornato

Casi d'uso correlati	EmettiBigliettoBiglietteria PrenotaBiglietto
Sequenza di eventi alternativi	Al punto 2 se il biglietto o la prenotazione è per autoveicolo 1. Il sistema aggiorna il numero di posti disponibili per autoveicoli.

Caso d'uso:	PrenotaBiglietto
Attore primario	Cliente
Attore secondario	-
Descrizione	Il cliente prenota un posto per la corsa specificata.
Pre-Condizioni	Il cliente deve essersi collegato al sito web della compagnia.
Sequenza di eventi principale	1. Il caso d'uso inizia quando il cliente richiede di prenotare un biglietto. 2. Il cliente specifica la tipologia del biglietto. 3. Il cliente inserisce il giorno, l'orario, porto di partenza e di arrivo della corsa. 4. Il sistema controlla che la corsa esista per i dati specificati. 4.1. Se la corsa non esiste il sistema restituisce un messaggio di ERRORE. 5. Include (VerificaDisponibilità). 6. Se il biglietto è per autoveicolo 6.1. Il cliente immette la targa dell'autoveicolo 7. Il sistema mostra al cliente il costo del biglietto della corsa. 8. Il cliente conferma la prenotazione ed effettua il pagamento 9. Il sistema genera una prenotazione per la corsa specificata. 10. Include (AggiornaNumeroPostiDisponibili). 11. Include (StampaPrenotazione).
Post-Condizioni	Il sistema ha generato una prenotazione.
Casi d'uso correlati	VerificaDisponibilità AggiornaNumeroPostiDisponibili StampaPrenotazione
Sequenza di eventi alternativi	-

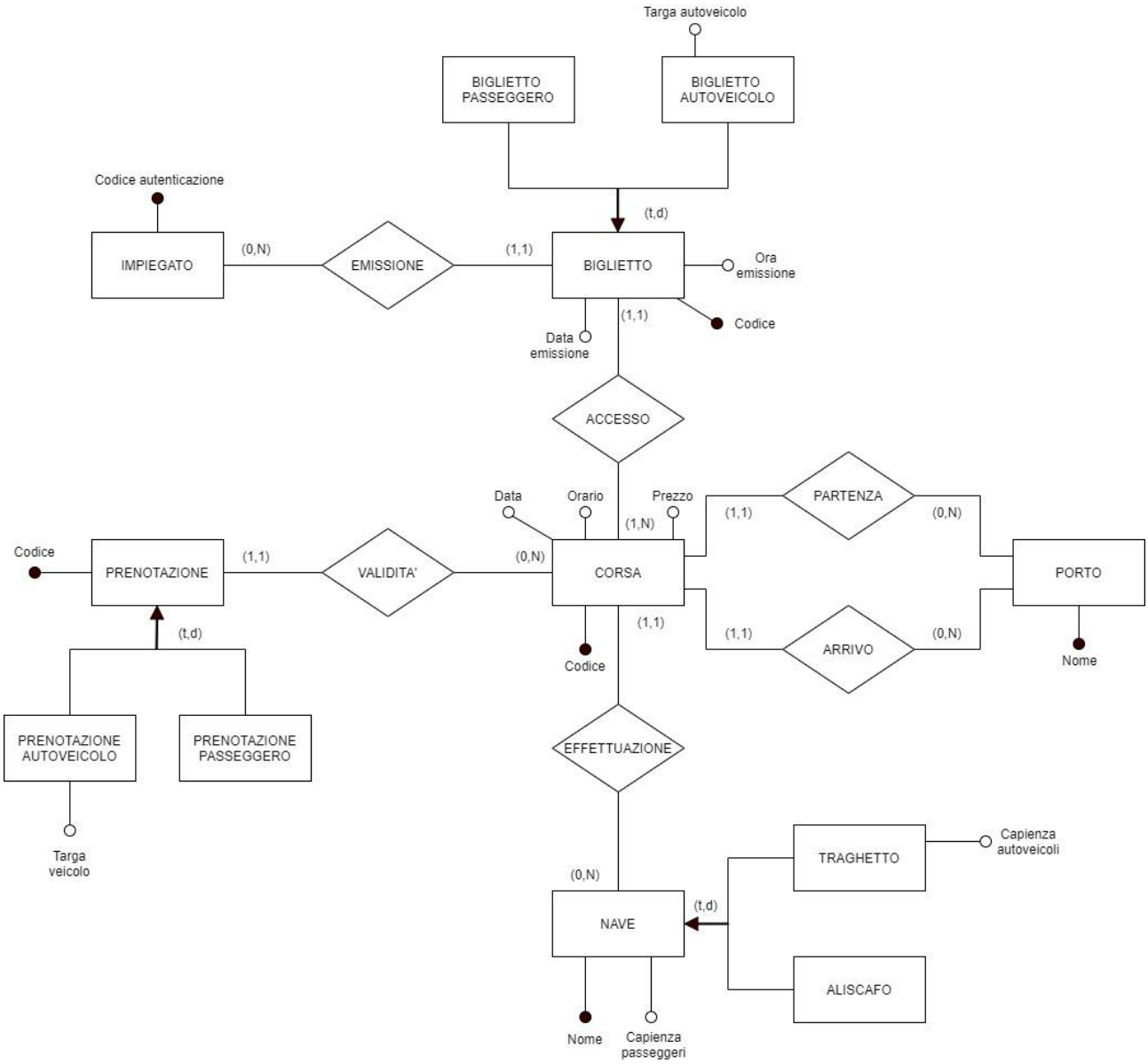
Caso d'uso:	StampaPrenotazione
Attore primario	-
Attore secondario	-
Descrizione	Viene stampata una prenotazione per una corsa
Pre-Condizioni	Il sistema deve aver generato una prenotazione
Sequenza di eventi principale	1. Il caso d'uso inizia quando il sistema ha aggiornato il numero di posti disponibili per la corsa 2. Il sistema effettua la stampa della prenotazione effettuata specificando il codice e gli estremi della corsa. 3. Se il biglietto è per autoveicolo 3.1. Il sistema stampa anche la targa dell'autoveicolo.
Post-Condizioni	Il sistema ha stampato una prenotazione per una corsa.
Casi d'uso correlati	PrenotaBiglietto
Sequenza di eventi alternativi	-

Caso d'uso:	VisualizzaCorse
Attore primario	Cliente
Attore secondario	-
Descrizione	Il Cliente visualizza l'insieme delle corse ancora disponibili
Pre-Condizioni	Il cliente deve essersi collegato al sito web della compagnia
Sequenza di eventi principale	1. Il caso d'uso inizia quando il cliente richiede di visualizzare le corse disponibili. 2. Il cliente inserisce il giorno della corsa 3. Il sistema mostra l'insieme delle corse disponibili per quel determinato giorno.
Post-Condizioni	-
Casi d'uso correlati	Nessuno
Sequenza di eventi alternativi	-

2.6 Modellazione dei dati

2.6.1 Progettazione concettuale

Diagramma E/R (Entità/Relazioni).



2.7 Diagramma delle classi

Diagramma delle classi di analisi

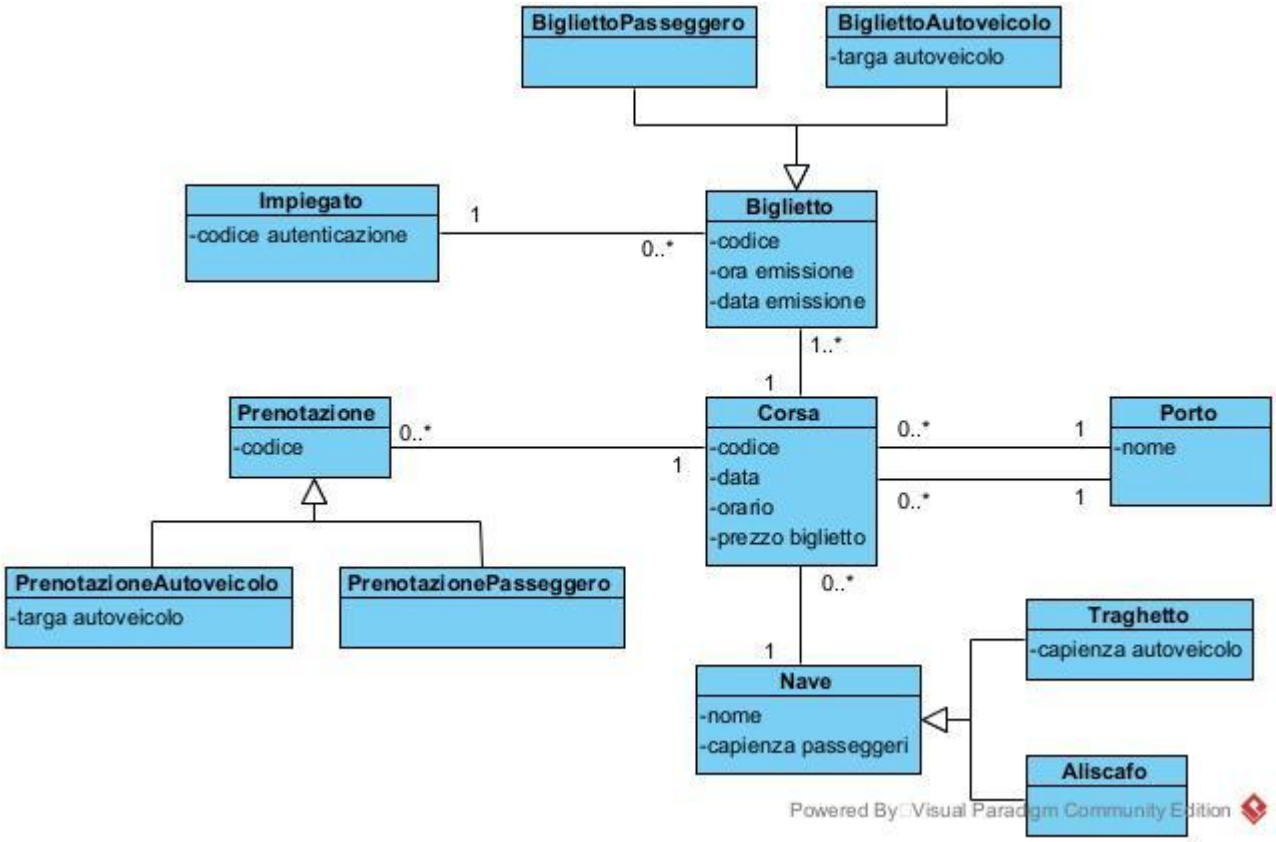
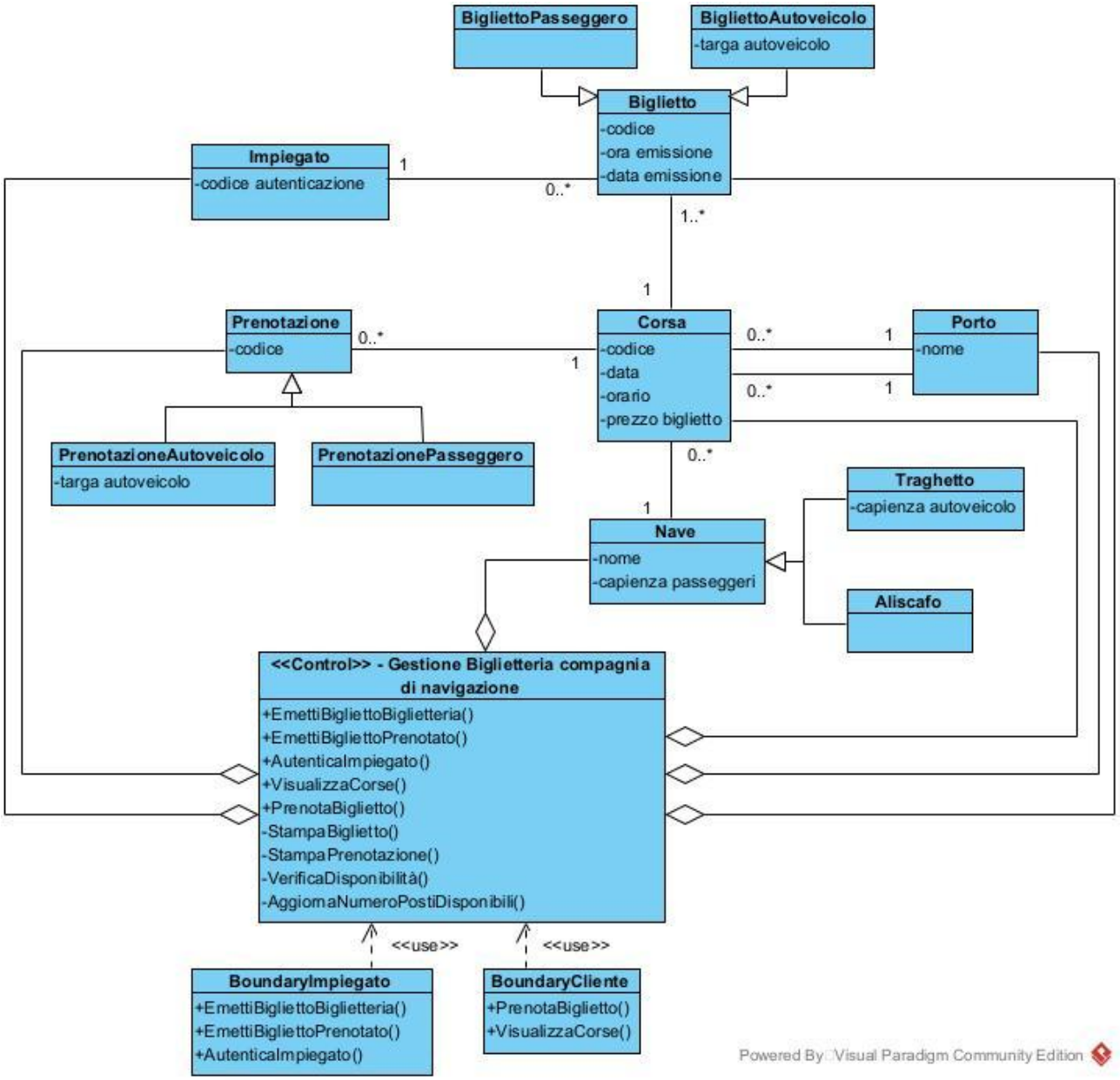
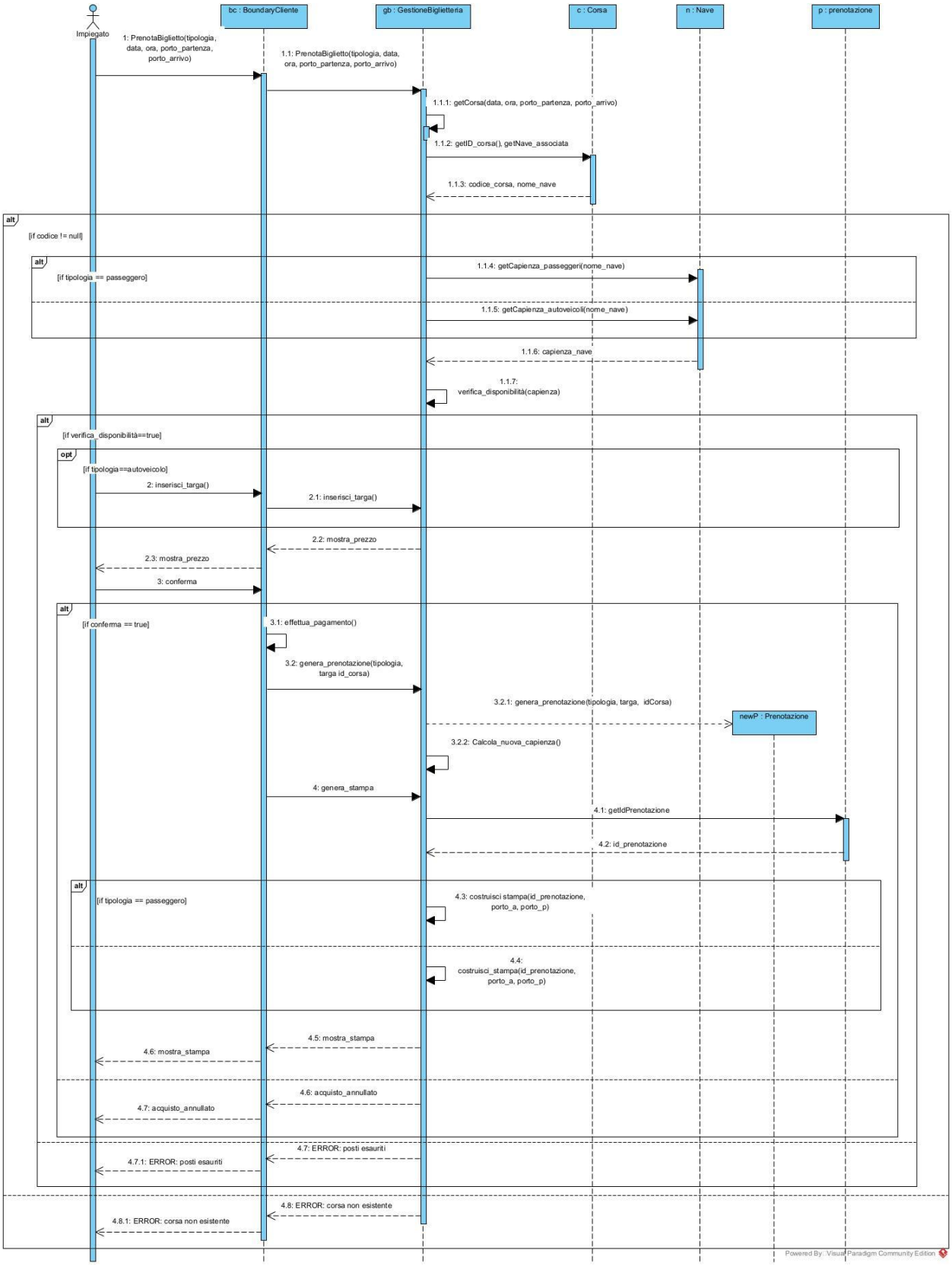


Diagramma delle classi raffinato (con classi Control e Boundary).



2.8 Diagrammi di sequenza

Diagramma di sequenza di analisi per il caso d'uso PrenotaBiglietto.



2.9 Verifica della completezza dei requisiti

Legenda: UCD = Use Case Diagram, CD = Class Diagram, SD = Sequence Diagram

- **RF01** è modellato nell'UCD con il caso d'uso UC7.
- **RF02** è modellato nell'UCD con l'attore "Cliente" e con il caso d'uso UC4, UC5.
- **RF03** è modellato nell'UCD con il caso d'uso UC9.
- **RF04** è modellato nell'UCD con l'attore "Impiegato" e con il caso d'uso UC3.
- **RF05** è modellato nell'UCD con l'attore "Impiegato" e con il caso d'uso UC1, UC2, UC3.
- **RF06** è modellato nell'UCD con l'attore "Impiegato" e con il caso d'uso UC1, UC2, UC3.
- **RF07** è modellato nell'UCD con il caso d'uso UC8.
- **RF08** è modellato nell'UCD con il caso d'uso UC1, UC2, UC3.
- **RF09** è modellato nell'UCD con il caso d'uso UC6.
- **RD01** è modellato nel CD con le classi "Traghetto" e "Aliscafo".
- **RD02** è modellato nel CD con la classe "Traghetto".
- **RD03** è modellato nel CD con gli attributi "nome" e "capienza passeggeri" della classe "Nave".
- **RD04** è modellato nel CD con l'attributo "capienza autoveicolo" della classe "Traghetto".
- **RD05** è modellato nel CD con l'associazione "uno a molti" tra le classi "Nave" e "Corsa" e con l'attributo "data" della classe "Corsa".
- **RD06** è modellato nel CD con l'associazione "uno a molti" tra le classi "Porto" e "Corsa".
- **RD07** è modellato nel CD con l'attributo "codice" della classe "Corsa".
- **RD08** è modellato nel CD con l'associazione "uno a molti" tra le classi "Porto" e "Corsa" e con gli attributi "ora" e "prezzo biglietto" della classe "Corsa".
- **RD09** è modellato nel CD con le classi "BigliettoPasseggero" e "BigliettoAutoveicolo".
- **RD10** è modellato nel CD con l'associazione "uno a molti" tra le classi "Corsa" e "Biglietto".

3. Stima dei costi

Si riporta la stima dei costi secondo il metodo dei Punti Funzione.

- Tabella di riferimento per le complessità di dati e transazioni

	SEMPLICE	MEDIO	COMPLESSO
NILF	7	10	15
NEIF	5	7	10
NEI	3	4	6
NEO	4	5	7
NEQ	3	4	8

CASO D'USO: UC4 - PrenotaBiglietto

	VALORE	SEMPLICE	MEDIO	COMPLESSO	TOTALE
NILF	1	-	10	-	10
NEIF	0	-	-	-	-
NEI	6	3	-	-	18
NEO	1	-	5	-	5
NEQ	0	-	-	-	-

NILF: La prenotazione viene creata dal sistema, la identifichiamo come ILF. [1 medio]
NEI: Tipologia Prenotazione, Data, Ora, Porto di partenza, Porto di arrivo, Targa. [6 semplici]
NEO: Stampa della Prenotazione, ottenuta tramite elaborazione [1 medio]

- Tabella elenco dei fattori correttivi (il cui valore è compreso tra 0 e 5).

FATTORI CORRETTIVI	
COMUNICAZIONE DATI	1
DISTRIBUZIONE ELABORAZIONE	0
PRESTAZIONI	4
UTILIZZO INTENSIVO CONFIGURAZIONE	2
FREQUENZA DELLE TRANSAZIONI	3
INSERIMENTO DATI INTERATTIVO	3
EFFICIENZA PER L'UTENTE FINALE	4
AGGIORNAMENTO INTERATTIVO	3
COMPLESSITÀ ELABORATIVA	0
RIUSABILITÀ	3
FACILITÀ INSTALLAZIONE	2

UFP	33
LLOC/FP	1749

FP	31,35
JAVA	1661

FACILITÀ GESTIONE OPERATIVA	2
MOLTEPLICITÀ DI SITI	0
FACILITÀ DI MODIFICA	3
	30

4. Piano di test funzionale

PIANO DI TEST CON IL METODO DEL *CATEGORY-PARTITION TESTING* PER LA FUNZIONALITÀ "*PrenotaBiglietto*".

TIPOLOGIA	DATA	ORA	PORTO PARTENZA	PORTO ARRIVO	TARGA	ELEMENTO DI SISTEMA DATABASE
• Stringa di caratteri = 'autoveicolo' o 'passeggero'. • Stringa di caratteri diversa da 'autoveicolo' o 'passeggero'.	• Data con formato valido (gg/mm/aaaa) • Data con formato non valido [ERROR]	• Ora con formato valido (hh:mm) • Ora con formato non valido [ERROR]	• Stringa di caratteri di lunghezza <= 100 • Stringa di caratteri di lunghezza > 100 [ERROR] • Stringa che contiene simboli [ERROR]	• Stringa di caratteri di lunghezza <= 100 • Stringa di caratteri di lunghezza > 100 [ERROR] • Stringa che contiene simboli [ERROR]	• Stringa di caratteri alfanumerici = 7 • Stringa di caratteri alfanumerici > 7 [ERROR] • Stringa di caratteri alfanumerici < 7 [ERROR] • Stringa che contiene simboli [ERROR]	• Capienza > 0 • Capienza <= 0 [ERROR]

Il numero di test da effettuarsi senza particolari vincoli è: 3 * 2 * 2 * 3 * 3 * 3 * 4 * 2 = 576
Introduciamo i vincoli [ERROR]
Il numero di test da eseguire per testare singolarmente i vincoli è (1 per Tipologia, 1 per Data, 1 per Ora, 2 per Porto Partenza, 2 per Porto Arrivo, 3 per Targa, 1 per ElementoDiSistemaDatabase).
Il numero di test risultante è: (1*1*1*1*1*1*1) + 11 = 12

Test Suite

Test Case ID	Descrizione	Classi di equivalenza coperte	Pre -condizioni	Input	Output attesi	Post-condizioni attese
1	Tutti input validi	Tipologia valida Data valida Ora valida Porto Partenza valido Porto Arrivo valido Targa valida ElementoDiSistemaDatabase valido	Il database è inizializzato correttamente. La corsa è in programma per la data, ora e i porti specificati. C'è la disponibilità del posto per effettuare una prenotazione di un biglietto.	{Tipologia: "Autoveicolo", Data: "04/06/2023", Ora: "10:30", Porto Partenza: "Ischia Porto", Porto Arrivo: "Pozzuoli" Targa: "DM089XA"}	Biglietto Prenotato	Il cliente riceve la stampa della prenotazione
2	Tipologia stringa diversa da 'autoveicolo' o 'passeggero'	Tipologia stringa non valida[ERROR] Data valida Ora valida Porto Partenza valido Porto Arrivo valido Targa valida ElementoDiSistemaDatabase valido	Il cliente ha selezionato la funzionalità per prenotare un biglietto. Il Sistema richiede all'utente di inserire la tipologia della prenotazione.	{Tipologia: "Autovei780oolo", Data: "04/06/2023", Ora: "10:30", Porto Partenza: "Ischia Porto", Porto Arrivo: "Pozzuoli" Targa: "DM089XA"}	Tipologia non valida	Il sistema chiede nuovamente di inserire la tipologia della prenotazione

3	Data formato non valido	Tipologia valida Data formato non valido [ERROR] Ora valida Porto Partenza valido Porto Arrivo valido Targa valida ElementoDiSistemaDatabase valido	Il cliente ha selezionato la funzionalità per prenotare un biglietto ed ha inserito la tipologia della prenotazione Il Sistema richiede all'utente di inserire la data.	{Tipologia: "Autoveicolo", Data: "68/532/091", Ora: "10:30", Porto Partenza: "Ischia Porto", Porto Arrivo: "Pozzuoli" Targa: "DM089XA"}	Data non valida	Il Sistema chiede nuovamente di inserire la data
4	Ora formato non valido	Tipologia valida Data valida Ora formato non valido [ERROR] Porto Partenza valido Porto Arrivo valido Targa valida ElementoDiSistemaDatabase valido	Il cliente ha selezionato la funzionalità per prenotare un biglietto ed ha inserito la tipologia della prenotazione e la data. Il Sistema richiede all'utente di inserire l'orario.	{Tipologia: "Autoveicolo", Data: "04/06/2023", Ora: "4589:3", Porto Partenza: "Ischia Porto", Porto Arrivo: "Pozzuoli" Targa: "DM089XA"}	Orario non valido	Il Sistema chiede nuovamente di inserire l'ora
5	Porto Partenza stringa > 100	Tipologia valida Data valida Ora valida Porto Partenza stringa > 100[ERROR] Porto Arrivo valido Targa valida ElementoDiSistemaDatabase valido	Il cliente ha selezionato la funzionalità per prenotare un biglietto ed ha inserito la tipologia della prenotazione, la data e l'ora. Il Sistema richiede all'utente di inserire il porto di partenza.	{Tipologia: "Autoveicolo", Data: "04/06/2023", Ora: "10:30", Porto Partenza: "Ischia Portooooooo

		ElementoDiSistemaDatabase valido	di partenza e di arrivo. Vi è disponibilità per la corsa specificata. Il Sistema richiede all'utente di inserire la targa.			
11	Targa stringa con simboli	Tipologia valida Data valida Ora valida Porto Partenza valido Porto Arrivo valido Targa stringa con simboli [ERROR] ElementoDiSistemaDatabase valido	Il cliente ha selezionato la funzionalità per prenotare un biglietto ha specificato "autoveicolo" come tipologia della prenotazione ed ha inserito la data, l'ora, il porto di partenza e di arrivo. Vi è disponibilità per la corsa specificata. Il Sistema richiede all'utente di inserire la targa.	{Tipologia: "Autoveicolo", Data: "04/06/2023", Ora: "10:30", Porto Partenza: "Ischia Porto", Porto Arrivo: "Pozzuoli" Targa: "DM089§A"}	Targa non valida	Il Sistema chiede nuovamente di inserire la targa
12	Capienza <= 0	Tipologia valida Data valida Ora valida Porto Partenza valido Porto Arrivo valido ElementoDiSistemaDatabase non valido	Non vi sono più posti disponibili per la corsa specificata	{Tipologia: "Autoveicolo", Data: "04/06/2023", Ora: "10:30", Porto Partenza: "Ischia Porto", Porto Arrivo: "Pozzuoli"}	Disponibilità posti esaurita	Non vi sono posti disponibili per la corsa specificata nel database

PIANO DI TEST CON IL METODO DEL *CATEGORY-PARTITION TESTING* PER LA FUNZIONALITÀ "Emetti*BigliettoPrenotato*".

CODICE PRENOTAZIONE	ELEMENTO DI SISTEMA DATABASE
• Stringa di caratteri di lunghezza <= 20 • Stringa di caratteri di lunghezza > 20 [ERROR] • Stringa che contiene simboli [ERROR]	• Prenotazione presente nel Database. • Prenotazione non presente nel database [ERROR]

Il numero di test da effettuarsi senza particolari vincoli è: 3 * 2 = 6
Introduciamo i vincoli [ERROR]
Il numero di test da eseguire per testare singolarmente i vincoli è (2 per Codice, 1 per ElementoDiSistemaDatabase).
Il numero di test risultante è: (1*1) + 3 = 4

Test Suite

Test Case ID	Descrizione	Classi di equivalenza coperte	Pre -condizioni	Input	Output attesi	Post-condizioni attese
1	Tutti input validi	Codice valido ElementoDiSistemaDatabase valido	Il database è inizializzato correttamente. Il codice del Biglietto è valido. Il codice della Prenotazione del biglietto è valido	{Codice: “0102335896”}	Stampa Biglietto	Il cliente riceve la stampa della prenotazione
2	Codice stringa > 20	Codice stringa > 20 [ERROR] ElementoDiSistemaDatabase valido	L’impiegato ha selezionato la funzionalità per emettere un biglietto. Il Sistema richiede il codice della prenotazione.	{Codice: “0102330000000000000000000005896”}	Codice non valido	Il sistema chiede nuovamente di inserire il codice della prenotazione
3	Codice stringa con simboli	Codice stringa con simboli [ERROR] ElementoDiSistemaDatabase valido	L’impiegato ha selezionato la funzionalità per emettere un biglietto. Il Sistema richiede il codice della prenotazione.	{Codice: “010233?????5896”}	Codice non valido	Il sistema chiede nuovamente di inserire il codice della prenotazione
4	Codice Prenotazione non presente nel database	Codice valido ElementoDiSistemaDatabase non valido	Non è presente il Codice Prenotazione nel Sistema	{Codice: “0102335896”}	Codice Prenotazione non valido	Il Sistema chiede nuovamente il codice

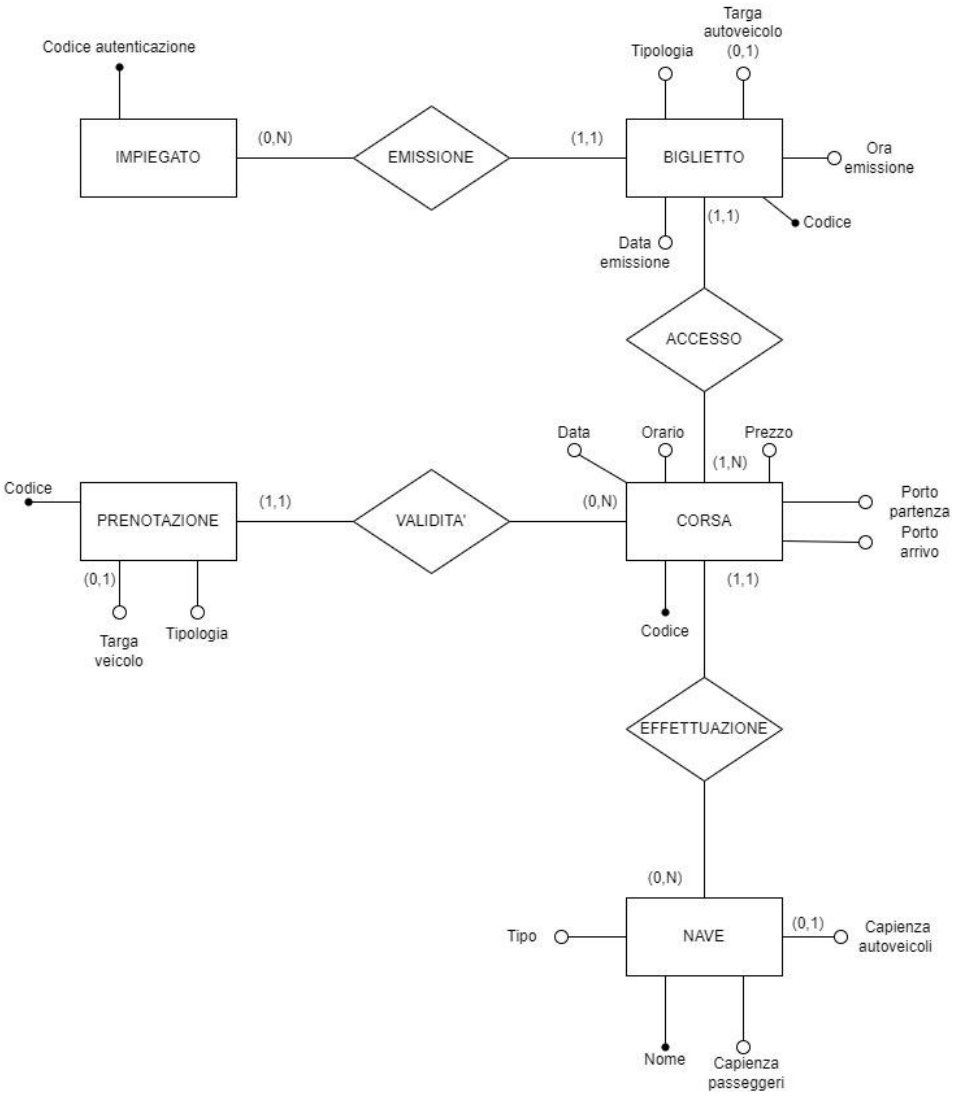
5. Progettazione

5.1 Progettazione della base di dati

5.1.1 Progettazione logica

È riportato di seguito lo schema relazionale, risultato della progettazione logica.
(Ad ogni entità viene associato un identificatore primario).
Si noti che la specializzazione delle classi Biglietto e Prenotazione viene risolta con l'aggiunta dell'attributo tipologia, l'associazione tra corsa e porto viene risolta aggiungendo gli attributi porto partenza e porto arrivo alla classe corsa, la specializzazione della classe Nave viene risolta con l'aggiunta dell'attributo tipo.

IMPIEGATO (CODICE AUTENTICAZIONE)
BIGLIETTO (CODICE, ORA_EMISSIONE, DATA_EMISSIONE, TIPOLOGIA, TARGA_VEICOLO, IMPIEGATO: IMPIEGATO, CORSA: CORSA)
PRENOTAZIONE (CODICE, TIPOLOGIA, TARGA_VEICOLO, CORSA: CORSA)
CORSA (CODICE, ORA, DATA, PREZZO, PORTO_PARTENZA, PORTO_ARRIVO, NAVE: NAVE)
NAVE (NOME, CAPIENZA_PASSEGGERI, CAPIENZA_AUTOVEICOLI, TIPO)



Si riportano i comandi SQL per la creazione dello schema della base di dati.

```
CREATE TABLE IMPIEGATI
(
        CODICE_AUTENTICAZIONE       INTEGER         AUTO_INCREMENT,
        PRIMARY KEY(CODICE_AUTENTICAZIONE)
);

CREATE TABLE BIGLIETTI
(
        CODICE                  INTEGER                                 AUTO_INCREMENT,
        ORA_EMISSIONE           TIME                                    NOT NULL,
        DATA_EMISSIONE          DATE                                    NOT NULL,
        TIPOLOGIA               ENUM('PASSEGGERO', 'AUTOVEICOLO')       NOT NULL,
        TARGA_VEICOLO           VARCHAR(7),
        IMPIEGATO               INTEGER,
        CORSA                   INTEGER,

        PRIMARY KEY(CODICE),
        FOREIGN KEY(IMPIEGATO) REFERENCES IMPIEGATI(CODICE_AUTENTICAZIONE),
        FOREIGN KEY(CORSA) REFERENCES CORSE(CODICE)
);

CREATE TABLE PRENOTAZIONI
(
        CODICE                  INTEGER                                 AUTO_INCREMENT,
        TIPOLOGIA               ENUM('PASSEGGERO', 'AUTOVEICOLO')       NOT NULL,
        TARGA_VEICOLO           VARCHAR(7),
        CORSA                   INTEGER,

        PRIMARY KEY(CODICE),
        FOREIGN KEY(CORSA) REFERENCES CORSE(CODICE)
);

CREATE TABLE CORSE
(
        CODICE                  INTEGER         AUTO_INCREMENT,
        ORA                     TIME            NOT NULL,
        DATA                    DATE            NOT NULL,
        PREZZO                  FLOAT(10),
        PORTO_PARTENZA          VARCHAR(100)    NOT NULL,
        PORTO_ARRIVO            VARCHAR(100)    NOT NULL,
        NAVE                    VARCHAR,

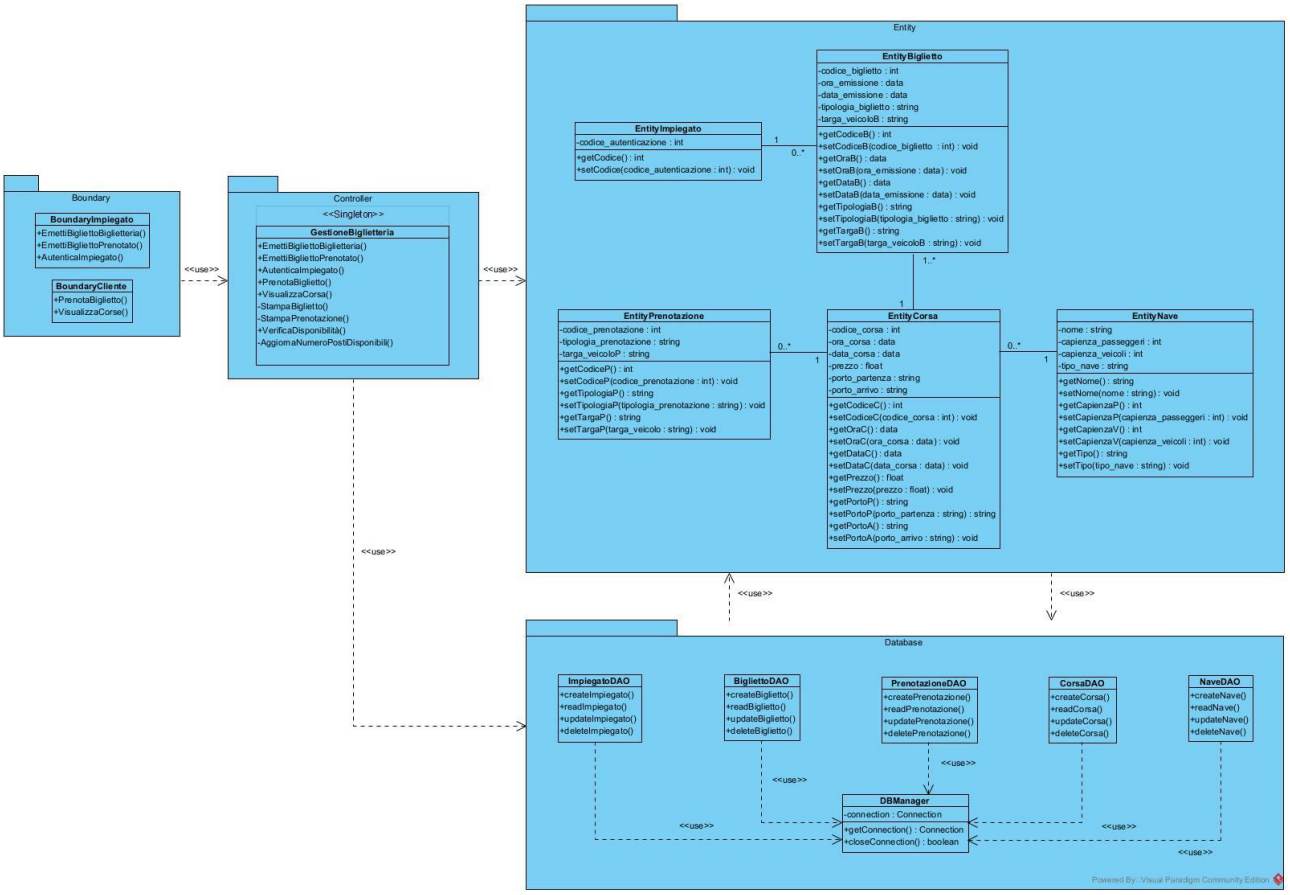
        PRIMARY KEY(CODICE),
        FOREIGN KEY(NAVE) REFERENCES NAVI(NOME)
);

CREATE TABLE NAVI
(
        NOME                    VARCHAR(30),
        CAPIENZA_PASSEGGERI     INTEGER                                 NOT NULL,
        CAPIENZA_AUTOVEICOLI    INTEGER,
        TIPO                    ENUM('ALISCAFO', 'TRAGHETTO')           NOT NULL,

        PRIMARY KEY(NOME)
);
```

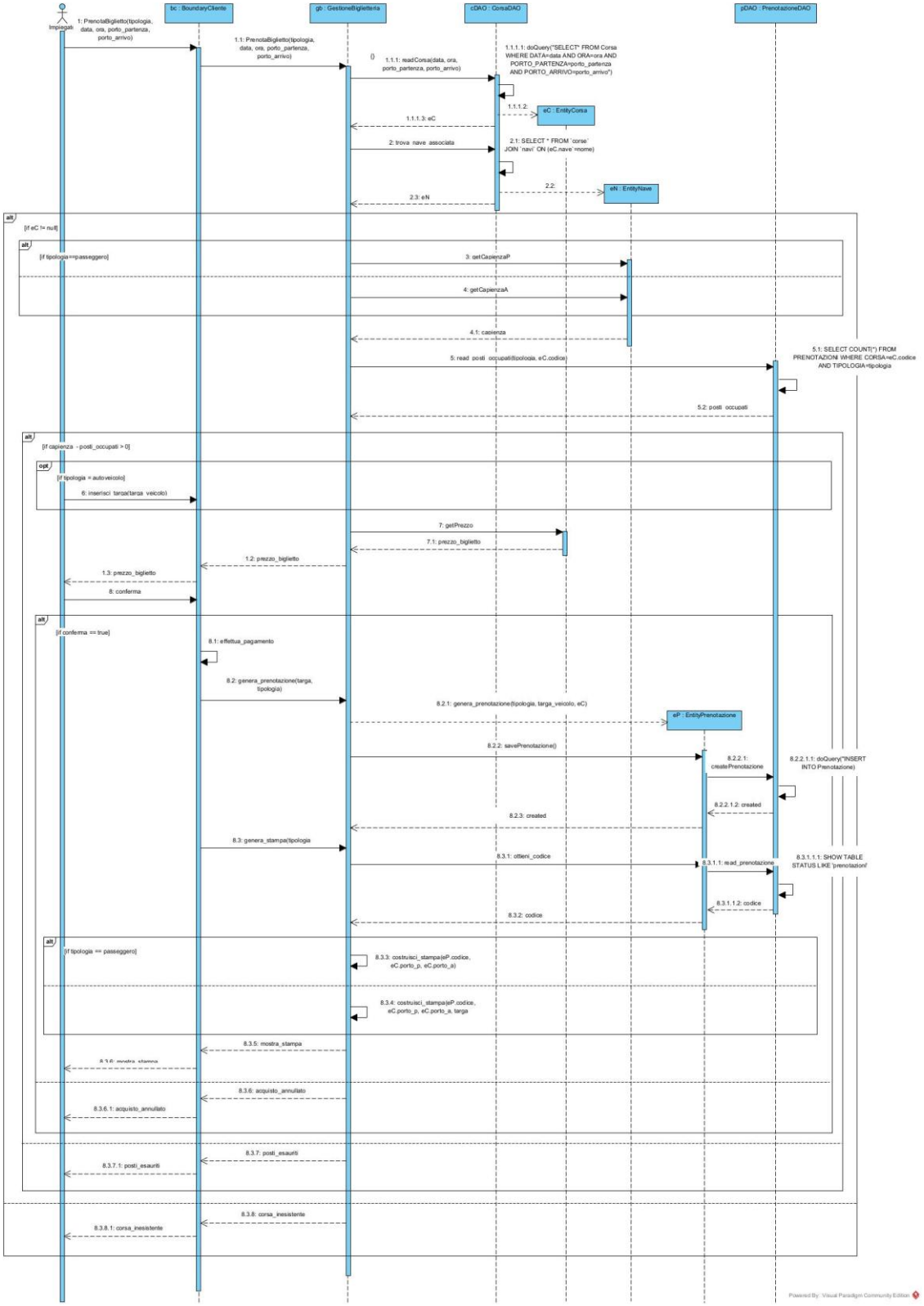
5.2 Diagramma delle classi

Diagramma delle classi di progettazione



5.3 Diagramma di sequenza

Si mostra il diagramma di sequenza di progettazione per il caso d'uso PrenotaBiglietto sviluppato fino alla codifica in Java.



6. Implementazione

Il package del progetto è strutturato nel seguente modo:

src

- boundary
  - BoundaryImpiegato
  - BoundaryCliente
- control
  - GestioneBiglietteria
- database
  - PrenotazioneDAO
  - CorsaDAO
  - NaveDAO
  - BigliettoDAO
  - ImpiegatoDAO
  - DBManager
- entity
  - EntityCorsa
  - EntityPrenotazione
  - EntityBiglietto
- exceptions
  - DAOExceptions
  - DBConnectionException
  - OperationException

Per l'istallazione è necessario il jar del dbms mysql.

7. Testing

7.1 Test Strutturale

7.1.1 Complessità ciclomatica

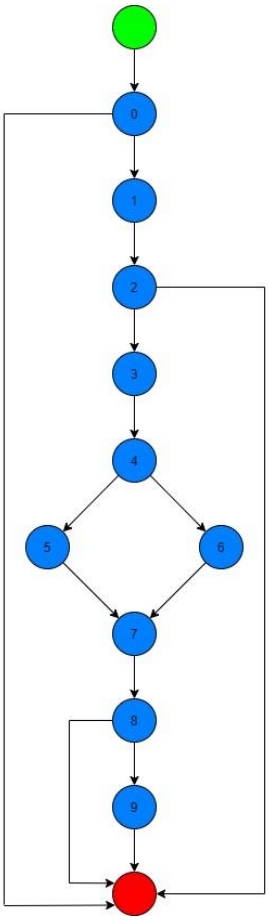
Si mostra il Control Flow Graph per il metodo PrenotaBiglietto() di una delle classi implementate, e:

- il calcolo del numero ciclomatico;

- i percorsi linearmente indipendenti;

```
public void prenotaBiglietto(String tipologia, Date data, Time ora, String porto_partenza, String porto_arrivo) throws OperationException{
		int capienza = 0;
		int posti_occupati = 0;

		try {
			eC = CorsaDAO.readCorsa(ora, data, porto_partenza, porto_arrivo);
			if(eC == null) {
				throw new OperationException("Corsa non trovata");
			} else {
				eC1 = new EntityCorsa(eC.getCodiceC());
				if (eC1.getNave_associata().getTipo().equals("ALISCAFO") && tipologia.equals("autoveicolo")) {
					throw new OperationException("Corsa non trovata");
				} else {
					System.out.println("Corsa trovata!");
					System.out.println("codice: " + eC1.getCodiceC());
					System.out.println("nome della nave: " + eC1.getNave_associata().getNome());
					if(tipologia.equals("passeggero")) {
						capienza = eC1.getNave_associata().getCapienzaP();
					} else {
						capienza = eC1.getNave_associata().getCapienzaA();
					}
					posti_occupati = PrenotazioneDAO.read_posti_occupati(tipologia, eC1.getCodiceC());
					if(capienza - posti_occupati <= 0) {
						throw new OperationException("Posti esauriti");
					}
					prezzo = eC1.getPrezzo();
				}
			}
		} catch (DBConnectionException dbEx) {
			throw new OperationException("\nRiscontrato problema interno applicazione!\n");
		} catch (DAOException ex) {
			throw new OperationException("Ops, qualcosa è andato storto..");
		}
}
```



Numero Ciclomatico:

- numero di regioni chiuse del grafo = 5
- numero di nodi predicati (0,2,4,8) +1 = 5
- # archi – # nodi + 2 = (15 - 11) + 2 = 6

Cammini:

1. 0
2. 0 – 1 – 2
3. 0 – 1 – 2 – 3 – 4 – 5 – 7 – 8
4. 0 – 1 – 2 – 3 – 4 – 6 – 7 – 8
5. 0 – 1 – 2 – 3 – 4 – 5 – 7 – 8 - 9
6. 0 – 1 – 2 – 3 – 4 – 6 – 7 – 8 - 9

7.1.2 Test di unità

Copertura dei cammini di base:

1. Inserimento di una corsa non valida o non presente nel database.
2. Inserimento tipologia = autoveicolo, corsa presente nel database ma effettua solo trasporto passeggeri.
3. Inserimento di una corsa valida con la tipologia del biglietto (autoveicolo)
4. Inserimento di una corsa valida con la tipologia del biglietto (passeggero)
5. Inserimento di una corsa valida con la tipologia del biglietto (autoveicolo) ma i posti sono esauriti.
6. Inserimento di una corsa valida con la tipologia del biglietto (passeggero) ma i posti sono esauriti.

7.2 Test funzionale

Sono descritti i risultati dell'esecuzione dei test funzionali precedentemente pianificati

Test Case ID	Descrizione	Classi di equivalenza coperte	Pre -condizioni	Input	Output attesi	Post-condizioni attese	Output ottenuti	Post-condizioni ottenute	Esito (FAIL/ PASS)
1	Tutti input validi	Tipologia valida Data valida Ora valida Porto Partenza valido Porto Arrivo valido Targa valida ElementoDiSistemaDatabase valido	Il database è inizializzato correttamente. La corsa è in programma per la data, ora e i porti specificati. C'è la disponibilità del posto per effettuare una prenotazione di un biglietto.	{Tipologia: "Autoveicolo", Data: "04/06/2023", Ora: "10:30", Porto Partenza: "Ischia Porto", Porto Arrivo: "Pozzuoli" Targa: "DM089XA"}	Biglietto Prenotato	Il cliente riceve la stampa della prenotazione	Biglietto prenotato!	Il cliente riceve la stampa della prenotazione	**PASS**
2	Tipologia stringa diversa da 'autoveicolo' o 'passeggero'	Tipologia stringa non valida[ERROR] Data valida Ora valida Porto Partenza valido Porto Arrivo valido Targa valida ElementoDiSistemaDatabase valido	Il cliente ha selezionato la funzionalità per prenotare un biglietto. Il Sistema richiede all'utente di inserire la tipologia della prenotazione.	{Tipologia: "Autovei780oolo", Data: "04/06/2023", Ora: "10:30", Porto Partenza: "Ischia Porto", Porto Arrivo: "Pozzuoli" Targa: "DM089XA"}	Tipologia non valida	Il sistema chiede nuovamente di inserire la tipologia della prenotazione	Tipologia non valida, riprovare...	Il sistema chiede nuovamente di inserire la tipologia della prenotazione	**PASS**
3	Data formato non valido	Tipologia valida Data formato non valido [ERROR] Ora valida Porto Partenza valido Porto Arrivo valido Targa valida ElementoDiSistemaDatabase valido	Il cliente ha selezionato la funzionalità per prenotare un biglietto ed ha inserito la tipologia della prenotazione Il Sistema richiede all'utente di inserire la data.	{Tipologia: "Autoveicolo", Data: "68/532/091", Ora: "10:30", Porto Partenza: "Ischia Porto", Porto Arrivo: "Pozzuoli" Targa: "DM089XA"}	Data non valida	Il Sistema chiede nuovamente di inserire la data	Data non valida, riprovare...	Il sistema chiede nuovamente di inserire la data	**PASS**
4	Ora formato non valido	Tipologia valida Data valida Ora formato non valido [ERROR] Porto Partenza valido Porto Arrivo valido Targa valida ElementoDiSistemaDatabase valido	Il cliente ha selezionato la funzionalità per prenotare un biglietto ed ha inserito la tipologia della prenotazione e la data. Il Sistema richiede all'utente di inserire l'orario.	{Tipologia: "Autoveicolo", Data: "04/06/2023", Ora: "4589:3", Porto Partenza: "Ischia Porto", Porto Arrivo: "Pozzuoli" Targa: "DM089XA"}	Orario non valido	Il Sistema chiede nuovamente di inserire l'ora	Orario non valido, riprovare...	Il sistema chiede nuovamente di inserire l'orario	**PASS**
5	Porto Partenza stringa > 100	Tipologia valida Data valida Ora valida Porto Partenza stringa > 100[ERROR] Porto Arrivo valido Targa valida ElementoDiSistemaDatabase valido	Il cliente ha selezionato la funzionalità per prenotare un biglietto ed ha inserito la tipologia della prenotazione, la data e l'ora. Il Sistema richiede all'utente di inserire il porto di partenza.	{Tipologia: "Autoveicolo", Data: "04/06/2023", Ora: "10:30", Porto Partenza: "Ischia Porto0000000000

11	Targa stringa con simboli	Tipologia valida Data valida Ora valida Porto Partenza valido Porto Arrivo valido Targa stringa con simboli [ERROR] ElementoDiSistemaDatabase valido	Il cliente ha selezionato la funzionalità per prenotare un biglietto ha specificato "autoveicolo" come tipologia della prenotazione ed ha inserito la data, l'ora, il porto di partenza e di arrivo. Vi è disponibilità per la corsa specificata. Il Sistema richiede all'utente di inserire la targa.	{Tipologia: "Autoveicolo", Data: "04/06/2023", Ora: "10:30", Porto Partenza: "Ischia Porto", Porto Arrivo: "Pozzuoli" Targa: "DM089§A"}	Targa non valida	Il Sistema chiede nuovamente di inserire la targa	Targa non valida, riprovare...	Il Sistema chiede nuovamente di inserire la targa	**PASS**
12	Capienza <= 0	Tipologia valida Data valida Ora valida Porto Partenza valido Porto Arrivo valido ElementoDiSistemaDatabase non valido	Non vi sono più posti disponibili per la corsa specificata	{Tipologia: "Autoveicolo", Data: "04/06/2023", Ora: "10:30", Porto Partenza: "Ischia Porto", Porto Arrivo: "Pozzuoli"}	Disponibilità posti esaurita	Non vi sono posti disponibili per la corsa specificata nel database	Posti esauriti	Non vi sono posti disponibili per la corsa specificata nel database	**PASS**